

SPIN - Seminar

Simple On-the-fly Automatic Verification
of Linear Temporal Logic

Formal Methods for AI-Based Systems Engineering (FMAISE)

Robert Li & William Tossici (1983696, 1844504)

Academic Year 2025/2026



SAPIENZA
UNIVERSITÀ DI ROMA



Table of Contents

1 Agenda

► Agenda

► Introduction

► Analysis

► Alternative Strategies

► Measurement

► The Hash-Factor

► Sequential Multihash

► Conclusion



Agenda: What We Will Cover

1 Agenda

- **The Core Problem:** Reachability analysis and hardware memory limits.
- **Bitstate Hashing (Supertrace):** Core concepts and mathematical foundations.
- **Alternative Strategies:** A critical comparison with Multihash and Hashcompact.
- **Empirical Measurements:** Validating theory with real protocols (DTP, PFTP).
- **Predicting Reliability:** The role of the "Hash-Factor".
- **Sequential Bitstate Hashing:** Pushing coverage limits in extreme memory constraints.



Table of Contents

2 Introduction

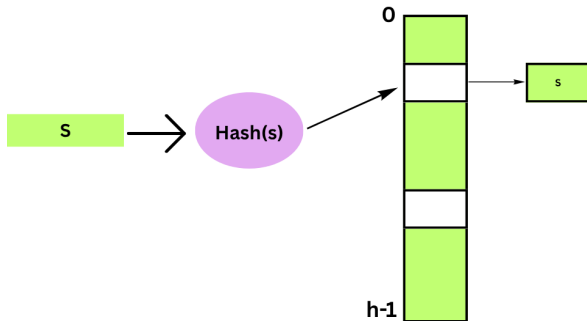
- ▶ Agenda
- ▶ **Introduction**
- ▶ Analysis
- ▶ Alternative Strategies
- ▶ Measurement
- ▶ The Hash-Factor
- ▶ Sequential Multihash
- ▶ Conclusion



How does SPIN store visited states?

2 Introduction

- Assume a hash table with h slots
- Each state is represented in memory as seq. of S bits
- **How to hash:**
 - **Slow way:** consider array of bits as one large unsigned integer and take modulo of div. by h
 - **Fast way (used in SPIN):** take h that is power of 2. Use a polynomial to compute a checksum of $\log(h)$ bits. Use the checksum as a hash value

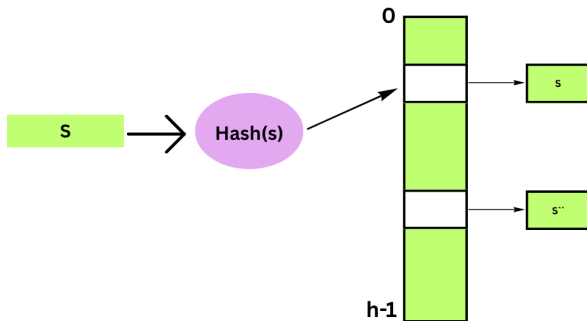




How does SPIN store visited states?

2 Introduction

- Assume a hash table with h slots
- Let r be #states stored in table
 - $h \gg r$: Ideal case. Each state can be stored in a different slot (given a good hash function). No time penalty.

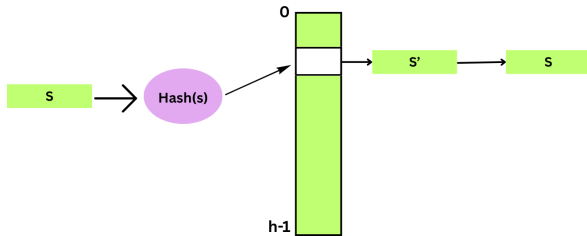




How does SPIN store visited states?

2 Introduction

- Assume a hash table with h slots
- Let r be #states stored in table
 - $h \gg r$: Ideal case. Each state can be stored in a different slot (given a good hash function). No time penalty.
 - $h > r$: Will be cases of hash collisions. They are resolved by storing states in a **linked list**. Towards the end of the search r/h comparisons will be required per state.
 - $h < r$: Overhead due to linked list search grows linearly with growing r/h





Introduction: The Need for Bitstate Hashing

2 Introduction

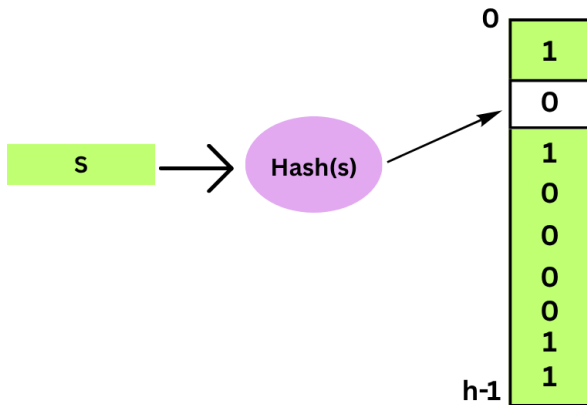
- **The Context:** Reachability analysis for huge concurrent systems.
- **The Solution:** Bitstate hashing (Supertrace) achieves high-coverage verification using orders of magnitude less memory.
- **Why not use Disk Storage?**
 - State exploration requires unpredictable, random read-accesses.
 - Disk access is too slow ($\sim 10^{-2}$ seconds per read).
 - Exploring 10^9 states on disk would take months of continuous computation.
 - **Conclusion:** The problem must be solved in Main Memory (RAM).



Bitstate Hashing

2 Introduction

- **The Blank Slate:** The system creates a massive array of M bits, all initialized to zero.
- Take state of size S . Hash it to get the hash value.
- Flip bit in the corresponding to hash value slot in the hash table.





Problem Definition: The Three Variables

2 Introduction

Key Parameters

- **N:** Total number of reachable states (*Unknown beforehand*).
 - **S:** Bits required to store a single state (*Known constant*).
 - **M:** Total available memory in bits (*Known hardware limit*).
-
- **The Crucial Metric:** The ratio M/N (memory available per state).
 - If $S \leq M/N$: Exhaustive search is possible.
 - **The Core Problem:** What happens when $S > M/N$? (i.e., Memory is strictly insufficient).



The Goal of Bitstate Hashing

2 Introduction

- **The Harsh Reality:** When $S > M/N$, we must store less information than the strict minimum required (S).
- **Consequence:** Loss of information is inevitable. No algorithm can guarantee 100% coverage in this domain.
- **The Ultimate Goal:**
 - Accept the unavoidable loss, but minimize it.
 - Maximize state coverage across the broadest possible range of M and N values, without assuming a cutoff limit.



Table of Contents

3 Analysis

- ▶ Agenda
- ▶ Introduction
- ▶ **Analysis**
- ▶ Alternative Strategies
- ▶ Measurement
- ▶ The Hash-Factor
- ▶ Sequential Multihash
- ▶ Conclusion



Analysis: Single-Bit Hashing ($h = 1$)

3 Analysis

- **Mathematical Setup:** We redefine variables as powers of two:

$$N = 2^n \quad M = 2^m \quad S = 2^s$$

- **The Procedure:** An array of 2^m bits is initialized to zero. Each state hashes to a single bit position.

Memory Array (M):

o	o	o	o	o	o	o	o
---	---	---	---	---	---	---	---

- Take state of size S . Hash it to get the hash value of m bits.
- Flip bit in the corresponding to hash value slot in the hash table.



Analysis: Single-Bit Hashing ($h = 1$)

3 Analysis

- **Mathematical Setup:** We redefine variables as powers of two:

$$N = 2^n \quad M = 2^m \quad S = 2^s$$

- **The Procedure:** An array of 2^m bits is initialized to zero. Each state hashes to a single bit position.

Memory Array (M):

o	o	o	o	o	o	o	o
---	---	---	---	---	---	---	---

- **If $N < M$:** Average probability of collision is bounded by $P_1 \leq 2^{n-m-1}$.
- **If $N \geq M$:** The array is physically too small. Minimum number of collisions is strictly $N - M$.
- **Lower Bound:** $P_1 \geq 1 - 2^{m-n}$ (Probability of collision drastically increases).



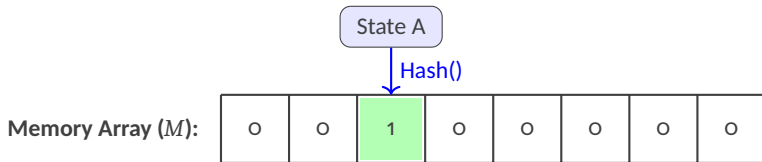
Analysis: Single-Bit Hashing ($h = 1$)

3 Analysis

- **Mathematical Setup:** We redefine variables as powers of two:

$$N = 2^n \quad M = 2^m \quad S = 2^s$$

- **The Procedure:** An array of 2^m bits is initialized to zero. Each state hashes to a single bit position.



- If $N < M$: Average probability of collision is bounded by $P_1 \leq 2^{n-m-1}$.
- If $N \geq M$: The array is physically too small. Minimum number of collisions is strictly $N - M$.
- **Lower Bound:** $P_1 \geq 1 - 2^{m-n}$ (Probability of collision drastically increases).



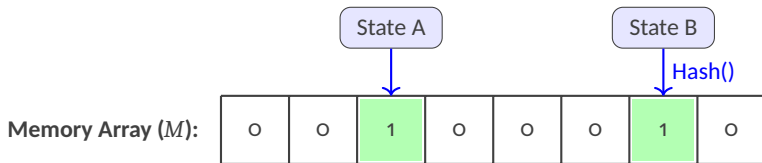
Analysis: Single-Bit Hashing ($h = 1$)

3 Analysis

- **Mathematical Setup:** We redefine variables as powers of two:

$$N = 2^n \quad M = 2^m \quad S = 2^s$$

- **The Procedure:** An array of 2^m bits is initialized to zero. Each state hashes to a single bit position.



- If $N < M$: Average probability of collision is bounded by $P_1 \leq 2^{n-m-1}$.
- If $N \geq M$: The array is physically too small. Minimum number of collisions is strictly $N - M$.
- **Lower Bound:** $P_1 \geq 1 - 2^{m-n}$ (Probability of collision drastically increases).



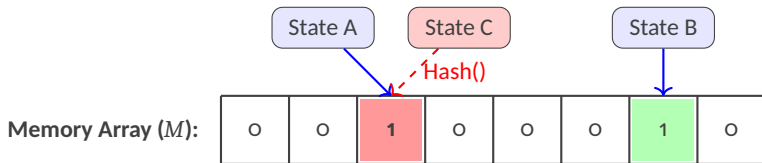
Analysis: Single-Bit Hashing ($h = 1$)

3 Analysis

- **Mathematical Setup:** We redefine variables as powers of two:

$$N = 2^n \quad M = 2^m \quad S = 2^s$$

- **The Procedure:** An array of 2^m bits is initialized to zero. Each state hashes to a single bit position.



- If $N < M$: Average probability of collision is bounded by $P_1 \leq 2^{n-m-1}$.
- If $N \geq M$: The array is physically too small. Minimum number of collisions is strictly $N - M$.
- **Lower Bound:** $P_1 \geq 1 - 2^{m-n}$ (Probability of collision drastically increases).

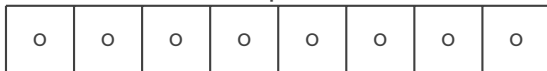


Analysis: Multi-Bit Hashing ($h \geq 2$)

3 Analysis

- **The Upgrade:** Using h independent hash functions per state.

Memory Array (M):



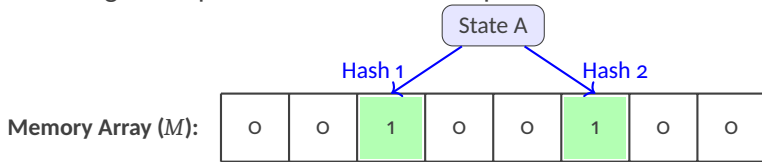
- If memory is sufficient ($2N < M$): $P_2 \leq \frac{3}{16} 2^{2(n-m-1)}$.
 - The collision probability is heavily reduced compared to single-bit hashing.
- **Approximation Error:** Analytical formulas are *pessimistic*. In well-connected systems, a truncated path might be reached via alternative routes, meaning fewer states are actually lost than predicted.



Analysis: Multi-Bit Hashing ($h \geq 2$)

3 Analysis

- **The Upgrade:** Using h independent hash functions per state.



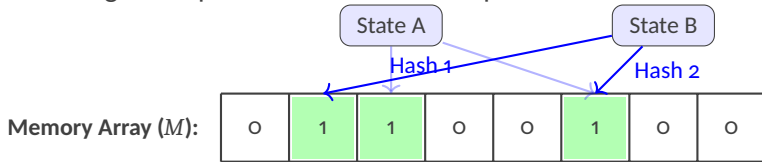
- If memory is sufficient ($2N < M$): $P_2 \leq \frac{3}{16} 2^{2(n-m-1)}$.
- The collision probability is heavily reduced compared to single-bit hashing.
- **Approximation Error:** Analytical formulas are *pessimistic*. In well-connected systems, a truncated path might be reached via alternative routes, meaning fewer states are actually lost than predicted.



Analysis: Multi-Bit Hashing ($h \geq 2$)

3 Analysis

- **The Upgrade:** Using h independent hash functions per state.



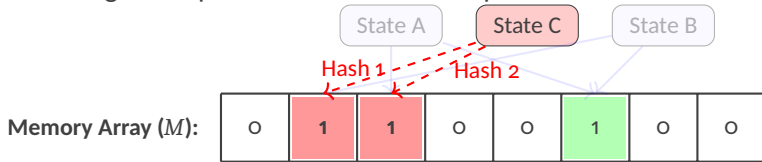
- If memory is sufficient ($2N < M$): $P_2 \leq \frac{3}{16} 2^{2(n-m-1)}$.
- The collision probability is heavily reduced compared to single-bit hashing.
- **Approximation Error:** Analytical formulas are *pessimistic*. In well-connected systems, a truncated path might be reached via alternative routes, meaning fewer states are actually lost than predicted.



Analysis: Multi-Bit Hashing ($h \geq 2$)

3 Analysis

- **The Upgrade:** Using h independent hash functions per state.



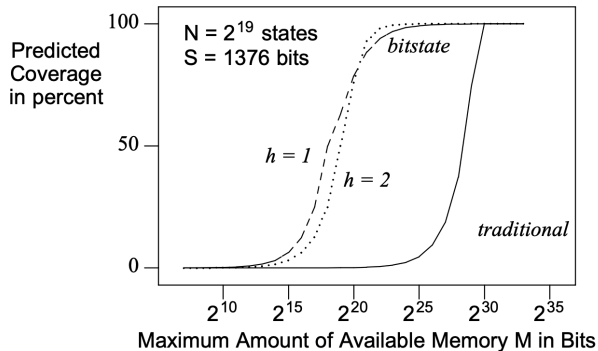
- If memory is sufficient ($2N < M$): $P_2 \leq \frac{3}{16} 2^{2(n-m-1)}$.
- The collision probability is heavily reduced compared to single-bit hashing.
- **Approximation Error:** Analytical formulas are *pessimistic*. In well-connected systems, a truncated path might be reached via alternative routes, meaning fewer states are actually lost than predicted.



Predicted Coverage: $h = 1$ vs $h = 2$

3 Analysis

- Exhaustive search drops to 0% coverage instantly when M shrinks.
- $h = 2$ is superior to $h = 1$ as long as $2N < M$.





Choosing the Optimal h

3 Analysis

- **What if we use $h = 20$?** Coverage improves for high M , but drops sharply for low M . It narrows the "optimal range" significantly.
- **Runtime Cost:** Verification time grows linearly with $h \cdot S$.
- **Conclusion:** $h = 2$ is the optimal compromise between run-time expense and high coverage for an unpredictable M/N ratio.

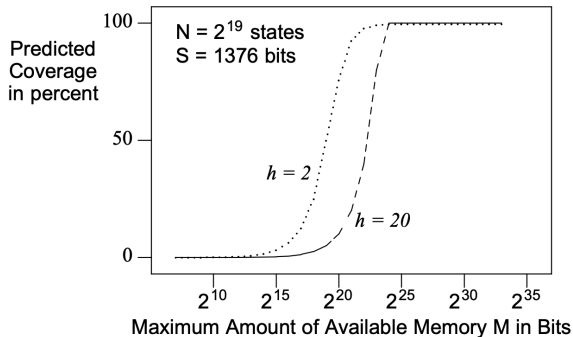




Table of Contents

4 Alternative Strategies

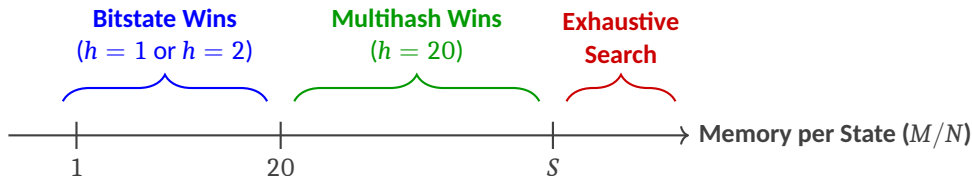
- ▶ Agenda
- ▶ Introduction
- ▶ Analysis
- ▶ **Alternative Strategies**
- ▶ Measurement
- ▶ The Hash-Factor
- ▶ Sequential Multihash
- ▶ Conclusion



Alternative 1: The Multihash Method ($h = 20$)

4 Alternative Strategies

Trade-off: 10x slower execution time for a narrow window of advantage.

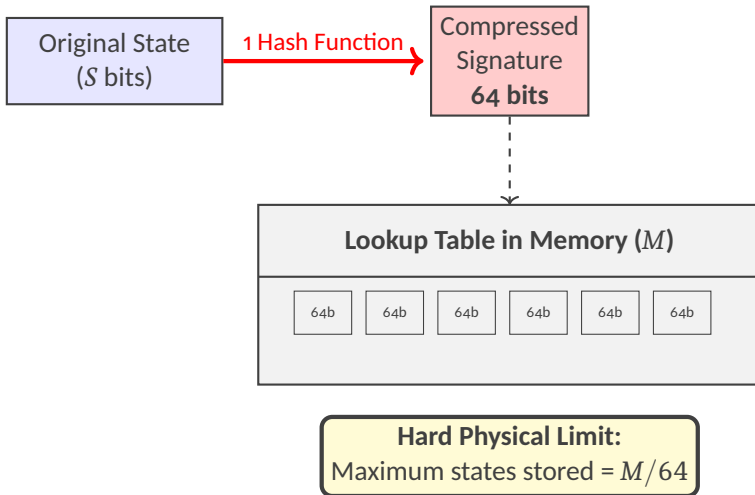


Since N is unknown beforehand, hitting the exact $20 \leq M/N \leq S$ target window is a gamble.



Alternative 2: The Hashcompact Method

4 Alternative Strategies





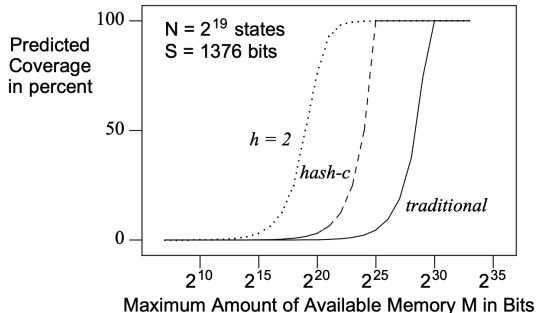
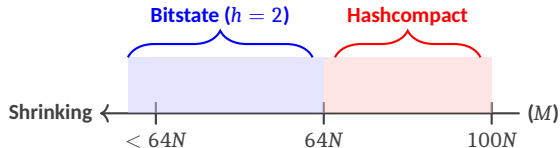
Hashcompact vs Bitstate ($h = 2$): The Sweet Spot

4 Alternative Strategies

When memory runs out (M shrinks):

Bitstate ($h = 2$)
Reliable up to $M/100$ states.
Degrades smoothly

Hashcompact
Reliable up to $M/64$ states.
Drops drastically



Hashcompact only wins in the tight window $64N \leq M \leq 100N$.
Outside this margin, $h = 2$ mathematically provides better coverage.



Table of Contents

5 Measurement

- ▶ Agenda
- ▶ Introduction
- ▶ Analysis
- ▶ Alternative Strategies
- ▶ **Measurement**
- ▶ The Hash-Factor
- ▶ Sequential Multihash
- ▶ Conclusion



Measurement: The "Optimal" Experiment (Table I)

5 Measurement

Initial test by Wolper & Leroy (DTP Protocol: 427,567 states)

Algorithm	Stored States	Memory Use	Missed?
Exhaustive	427,567	73 Mbytes	No
Bitstate	427,446	33.8 Mbytes	Yes
Hashcompact	427,567	3.6 Mbytes	No



Measurement: The "Optimal" Experiment (Table I)

5 Measurement

Initial test by Wolper & Leroy (DTP Protocol: 427,567 states)

Algorithm	Stored States	Memory Use	Missed?
Exhaustive	427,567	73 Mbytes	No
Bitstate	427,446	33.8 Mbytes	Yes
Hashcompact	427,567	3.6 Mbytes	No

The Catch:

3.6 Mbytes $\approx 450,000 \times 64$ bits.

This test sets exactly $M \approx 64N$.

A perfect, but unrealistic, scenario.



Measurement: The Full Domain (Table II)

5 Measurement

Holzmann's Test: Progressively shrinking the memory (M)

Memory (M)	Bitstate ($h = 2$)	Hashcompact
26.84 Mbytes	99.9%	100%
4.19 Mbytes	99.9%	100%

----- $M < 64N$



Measurement: The Full Domain (Table II)

5 Measurement

Holzmann's Test: Progressively shrinking the memory (M)

Memory (M)	Bitstate ($h = 2$)	Hashcompact
26.84 Mbytes	99.9%	100%
4.19 Mbytes	99.9%	100%
2.09 Mbytes	99.9%	61.3%

----- $M < 64N$



Measurement: The Full Domain (Table II)

5 Measurement

Holzmann's Test: Progressively shrinking the memory (M)

Memory (M)	Bitstate ($h = 2$)	Hashcompact
26.84 Mbytes	99.9%	100%
4.19 Mbytes	99.9%	100%
----- $M < 64N$ -----		
2.09 Mbytes	99.9%	61.3%
1.05 Mbytes	99.5%	30.6%



Measurement: The Full Domain (Table II)

5 Measurement

Holzmann's Test: Progressively shrinking the memory (M)

Memory (M)	Bitstate ($h = 2$)	Hashcompact
26.84 Mbytes	99.9%	100%
4.19 Mbytes	99.9%	100%
----- $M < 64N$ -----		
2.09 Mbytes	99.9%	61.3%
1.05 Mbytes	99.5%	30.6%
0.26 Mbytes	85.7%	7.7%

Conclusion: Bitstate gracefully degrades to 85.7% coverage. Hashcompact completely collapses to 7.7%.



Table of Contents

6 The Hash-Factor

- ▶ Agenda
- ▶ Introduction
- ▶ Analysis
- ▶ Alternative Strategies
- ▶ Measurement
- ▶ **The Hash-Factor**
- ▶ Sequential Multihash
- ▶ Conclusion

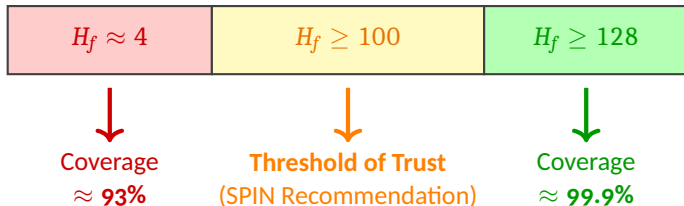


Predicting Reliability: The Hash-Factor

6 The Hash-Factor

The Problem: How do we know the actual coverage if an exhaustive search is impossible?

The Solution: SPIN uses a predictor function called **Hash-Factor** (H_f)



The Hash-Factor tells the user a posteriori if the verification was reliable.



Hash-Factor vs Actual Coverage

6 The Hash-Factor

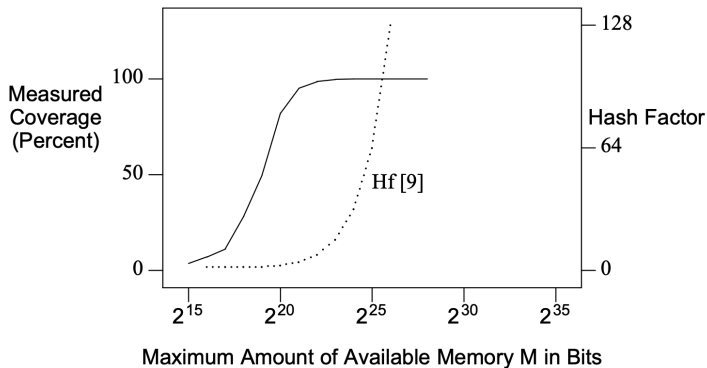
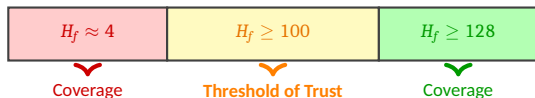




Table of Contents

7 Sequential Multihash

- ▶ Agenda
- ▶ Introduction
- ▶ Analysis
- ▶ Alternative Strategies
- ▶ Measurement
- ▶ The Hash-Factor
- ▶ **Sequential Multihash**
- ▶ Conclusion



The Ultimate Evolution: Sequential Multihash

7 Sequential Multihash

The Concept: Instead of running h functions simultaneously, run the bitstate algorithm *repeatedly*, using a different, independent hash function each time.

The Mathematical Power

- If the probability of collision in one run is p ($p < 1$).
- After h sequential runs, the probability of missing a state becomes p^h .
- As $h \rightarrow \infty$, $p^h \rightarrow 0$.
- **The Trade-off:** We trade *time* (running the program h times) to compensate for the severe lack of *memory*.
- **Implementation in SPIN:** Uses CRC polynomials. SPIN 2 provides up to 32 independent polynomials.



Sequential Performance: Theory vs Reality

7 Sequential Multihash

Theoretical Runs Needed for $\sim 100\%$ Coverage

Initial p_1	Runs (h)
0.01	6
0.30	11
0.51	19
0.80	59
0.90	124

The "Ideal Circumstances" Requirement

- Requires a **well-connected** state graph.
- If states are strictly on a single path, a collision blocks everything.
- Fortunately, asynchronous concurrent systems naturally produce highly connected graphs.



Measurement: The DFS Penalty and Randomization

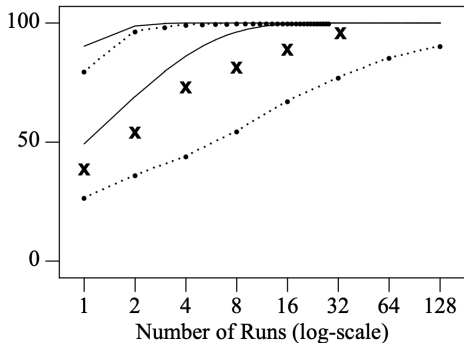
7 Sequential Multihash

- **The Depth-First Search (DFS) Penalty:** Software like SPIN does not discover states randomly, but uses ordered DFS. This lowers real coverage compared to ideal predictions.
- **The Extreme Test (32 Kb Memory):**
 - After 128 runs, coverage reached 90% (not 100%).
 - **Yet, a computational miracle:** The tool explored **386,398 states** using an arena of only **262,144 bits**.
- **The Fix:** Randomizing the DFS order brings coverage back close to the theoretical maximum.



Measurement: The DFS Penalty and Randomization

7 Sequential Multihash



Graph Legend:

- Solid Lines (Ideal)**
Randomly generated states. Shows theoretical max coverage.
- Dotted Lines (Reality)**
Normal DFS order. Sequential generation lowers coverage.
- 'x' Marks (The Fix)**
Randomized DFS. Pushes coverage back toward the ideal curve.

*Note: Top curves represent 256Kb memory.
Bottom curves represent 32Kb memory.*



Table of Contents

8 Conclusion

- ▶ Agenda
- ▶ Introduction
- ▶ Analysis
- ▶ Alternative Strategies
- ▶ Measurement
- ▶ The Hash-Factor
- ▶ Sequential Multihash
- ▶ **Conclusion**



Conclusions

8 Conclusion

- **Bitstate is Robust:** It makes fewer assumptions about the application and outperforms competitors for large problems or severe memory constraints.
- **When to use Compression (Hashcompact):** Only if a prior bitstate run reveals that $M/2 < N < M/S$.
- **The Ultimate Solution:** Sequential bitstate hashing can outperform all other methods for extremely large problems by trading time for memory.
- **Final Thought:** No method removes the fundamental computational complexity of the state explosion problem. We simply manage it.